

ASG Airlines - Data Pipeline

This notebook builds an end-to-end pipeline for the ASG Airlines use case: ingest the raw data, clean it, protect sensitive passenger information, and produce a data model ready for Power BI.

The written problem statement only describes the flights table's columns, but the actual file provided has 4 related sheets - flights, bookings, payments, and passengers - connected through flight_id, booking_id, and passenger_id. Since all 4 were part of the given data, all 4 are used here, not just flights.

Overall structure, using a standard bronze/silver/gold layering:

- **Bronze:** raw data, exactly as given, no changes
- **Silver:** cleaned data, one issue fixed at a time, each one diagnosed first and then fixed
- **Gold:** a proper star schema (fact_bookings + dim_flight + dim_passenger) plus one flattened table for building the Power BI dashboard easily

Step 1: Ingestion (Bronze layer)

Reading the 4 raw CSV files from a Unity Catalog Volume. Storage-key based access to ADLS directly wasn't available on serverless compute (a governance restriction, not something broken), so the files were uploaded into a Volume instead, which is Databricks' own recommended way to bring in files under Unity Catalog.

```
In [0]: base_path = "/Volumes/asg_airlines_databricks/default/raw_data/raw_csv_for_u

flights = spark.read.csv(base_path + "/flights.csv", header=True, inferSchem
bookings = spark.read.csv(base_path + "/bookings.csv", header=True, inferSch
payments = spark.read.csv(base_path + "/payments.csv", header=True, inferSch
passengers = spark.read.csv(base_path + "/passengers.csv", header=True, infe

flights.count(), bookings.count(), payments.count(), passengers.count()
```

```
Out[0]: (1020, 1000, 1000, 1039)
```

```
In [0]: from pyspark.sql import functions as F
from pyspark.sql.window import Window
```

Step 2: Cleaning the flights table (Silver layer)

The problem statement mentions the flights data has issues like corrupted identifiers, missing values and time inconsistencies. Instead of assuming what's wrong, checking

the actual data first, then fixing each issue one at a time.

2a. Exact duplicate rows

```
In [0]: total_rows = flights.count()
distinct_rows = flights.distinct().count()

print("total rows:", total_rows)
print("distinct rows:", distinct_rows)
print("exact duplicate rows:", total_rows - distinct_rows)
```

```
total rows: 1020
distinct rows: 1005
exact duplicate rows: 15
```

Found 15 rows that are exact duplicates - every single column value matches another row. This is likely the same flight event getting logged twice somewhere upstream (booking platform + scheduling system both reporting it, maybe). Since they carry zero new information and would inflate route-wise traffic counts if left in, dropping them.

```
In [0]: flights_clean = flights.dropDuplicates()
flights_clean.count()
```

```
Out[0]: 1005
```

2b. Same flight_id used for two different flights

A flight_id should be unique to one flight. Checking if any flight_id shows up more than once, even after removing exact duplicates.

```
In [0]: dup_ids = flights_clean.groupBy("flight_id").count().filter("count > 1")
dup_ids.show()
```

```
+-----+-----+
|flight_id|count|
+-----+-----+
|    6F250|    2|
+-----+-----+
```

```
In [0]: flights_clean.filter(flights_clean.flight_id == "6F250").show()
```

flight_id	airline	source	destination	departure_time	arrival_time	duration
6F250	UNKNOWN	DEL	BLR	2026-04-20 03:26:...	2026-04-20 07:30:...	2026-09-05 04:04:00
6F250	UNKNOWN	CCU	BLR	2026-04-20 02:23:...	2026-04-20 02:56:...	2026-09-05 00:33:00

Both 6F250 rows are real, different flights - different route, different time. They just happen to share the same flight_id, most likely a system error where the same ID got assigned twice.

Not deleting either one since both are real flights with real data. Instead, adding a new column that gives every row its own unique number, so flights can always be told apart even when flight_id repeats. Keeping the original flight_id column too, since it's still useful for reporting - just noting this as a known data issue in the source system.

```
In [0]: flights_clean = flights_clean.withColumn("flight_row_id", F.monotonically_increasing_id())
        flights_clean.select("flight_row_id", "flight_id", "source", "destination").
```

flight_row_id	flight_id	source	destination
0	UK067	BOM	CCU
1	UK200	CCU	DEL
2	SJ043	MAA	DEL
3	UK152	BOM	CCU
4	SJ081	BOM	BLR

only showing top 5 rows

2c. Impossible flight duration

A flight can't arrive before it leaves - even for overnight flights that land the next day, time is still moving forward. Checking for any flight where arrival minus departure comes out negative.

```
In [0]: flights_clean = flights_clean.withColumn(
        "duration_minutes",
        (F.unix_timestamp("arrival_time") - F.unix_timestamp("departure_time"))
    )

        flights_clean.filter(flights_clean.duration_minutes < 0).show()
```

```

+-----+-----+-----+-----+-----+-----+
|flight_id| airline|source|destination| departure_time| arrival_time|
|         |         |       |            | duration      | flight_row_id|
+-----+-----+-----+-----+-----+-----+
|    SJ192|SpiceJet|  HYD|    BOM|2026-04-19 18:45:42|2026-04-18 23:45:42|
|1899-12-29 05:00:00|         |456|         -1140.0|
+-----+-----+-----+-----+-----+

```

Found one flight, SJ192, where arrival time is about 19 hours before departure time. This isn't a normal overnight flight (those still move forward in time, just cross midnight) - this is a bad record, maybe departure and arrival got swapped or logged wrong at the source. Since the times on this one can't be trusted, removing it from the clean dataset rather than guessing at a fix, but keeping a note of it separately so it's not silently lost.

```

In [0]: invalid_flights = flights_clean.filter(flights_clean.duration_minutes < 0)
        flights_clean = flights_clean.filter(flights_clean.duration_minutes >= 0)

        print("invalid rows removed:", invalid_flights.count())
        print("remaining rows:", flights_clean.count())

```

```

invalid rows removed: 1
remaining rows: 1004

```

2d. Two different ways of saying "airline unknown"

```

In [0]: flights_clean.groupBy("airline").count().show()

```

```

+-----+-----+
| airline|count|
+-----+-----+
| Vistara|  218|
| UNKNOWN|   30|
| SpiceJet| 235|
|   NULL |   39|
|  IndiGo|  249|
| Air India| 233|
+-----+-----+

```

Blank/null airline and the literal text "UNKNOWN" are being counted separately, even though they mean the same thing. Having two different labels for the same "missing" state would mess up the airline distribution numbers later, so combining them into one.

```

In [0]: flights_clean = flights_clean.fillna({"airline": "UNKNOWN"})
        flights_clean.groupBy("airline").count().show()

```

airline	count
Vistara	218
UNKNOWN	69
SpiceJet	235
IndiGo	249
Air India	233

2e. Flagging overnight flights

The problem statement specifically calls out overnight (cross-day) flights as something that needs to be handled properly. Adding this as its own column rather than leaving it buried inside the duration number, so it can be used directly on the dashboard.

Note: the duration itself (calculated above using full timestamps, not just time-of-day) is already correct for these flights, since it accounts for the date change automatically. This flag is just for visibility, not fixing a calculation error.

```
In [0]: flights_clean = flights_clean.withColumn(
    "is_overnight",
    F.to_date("arrival_time") > F.to_date("departure_time")
)

flights_clean.groupBy("is_overnight").count().show()
```

is_overnight	count
true	122
false	882

Step 3: Cleaning the bookings table

Checking the status column - a booking should always have a clear status.

```
In [0]: bookings.groupBy("status").count().show()
```

status	count
CONFIRMED	320
INVALID	30
PENDING	291
NULL	45
CANCELLED	314

Two problems here: some bookings have no status at all (blank), and separately there's a status called "INVALID" which isn't one of the normal booking states. Treating blank/null status as "UNKNOWN" (same approach as the airline column), but keeping "INVALID" as its own separate category rather than merging it in - it likely means the source system itself flagged that booking as broken, which is different from the status simply not being recorded.

```
In [0]: bookings_clean = bookings.fillna({"status": "UNKNOWN"})
bookings_clean.groupBy("status").count().show()
```

```
+-----+-----+
|  status | count |
+-----+-----+
| CONFIRMED | 320 |
|  INVALID | 30 |
|  PENDING | 291 |
|  UNKNOWN | 45 |
| CANCELLED | 314 |
+-----+-----+
```

Step 4: Cleaning the payments table

4a. Missing amounts

```
In [0]: payments.filter(payments.amount.isNull()).count()
```

```
Out[0]: 48
```

Found 48 payments with no amount recorded. Not filling these in with 0 or a guessed value, since that would make it look like the payment was actually zero or an average amount, which isn't true - it's just not known. Keeping these rows as they are, and only summing the payments that actually have an amount when calculating revenue later.

4b. Non-numeric junk in the amount column

```
In [0]: payments.filter(payments.amount == "INVALID").count()
```

```
Out[0]: 30
```

Found 30 rows where amount is the literal text "INVALID" rather than a number - the same underlying problem as the blank amounts, just showing up as text instead of null. Converting these to proper nulls using `try_cast`, which turns unparseable values into null instead of crashing the calculation.

```
In [0]: payments = payments.withColumn("amount", F.expr("try_cast(amount as double)"))
payments.filter(payments.amount.isNull()).count()
```

Out [0]: 78

4c. Some bookings have more than one payment record

Assumed each booking would have at most one payment - checking that assumption before joining payments in.

```
In [0]: payment_counts = payments.groupBy("booking_id").count()
payment_counts.groupBy("count").count().orderBy("count").show()
```

```
+-----+-----+
|count|count|
+-----+-----+
|    1|   370|
|    2|   189|
|    3|    63|
|    4|    13|
|    5|     1|
|    6|     1|
+-----+-----+
```

Found that some bookings have multiple payment records - up to 6 for a single booking, with different amounts and different payment methods. There's no status or timestamp field to tell whether these are separate legitimate charges (like a split payment) or failed payment attempts that got logged anyway.

This is an assumption worth stating explicitly: treating all payment records for a booking as real amounts and summing them into a total, rather than guessing which one to keep and discarding the rest. Also adding a flag for bookings with more than one payment record, so this stays visible on the dashboard as something worth investigating operationally, instead of disappearing during cleaning.

```
In [0]: payments_agg = payments.groupBy("booking_id").agg(
    F.sum("amount").alias("total_amount"),
    F.count("payment_id").alias("payment_count")
).withColumn("multiple_payment_attempts", F.col("payment_count") > 1)

payments_agg.show(5)
```

```
+-----+-----+-----+-----+
|booking_id|total_amount|payment_count|multiple_payment_attempts|
+-----+-----+-----+-----+
|    B1998|    18385.6|          2|                true|
|    B1736|    8590.58|          2|                true|
|    B1641|    14953.42|          1|               false|
|    B1572|    13849.86|          2|                true|
|    B1459|    19085.7|          3|                true|
+-----+-----+-----+-----+
```

only showing top 5 rows

Step 5: Cleaning the passengers table

5a. Missing last names

```
In [0]: passengers.filter(passengers.last_name.isNull()).count()
```

```
Out[0]: 10
```

Found 10 passengers with no last name recorded. This doesn't affect any of the KPIs, so instead of guessing a name, filling it with "Unknown" so the field isn't blank, and it's clearly visible the name wasn't available rather than silently missing.

```
In [0]: passengers_clean = passengers.fillna({"last_name": "Unknown"})
passengers_clean.filter(passengers_clean.last_name == "Unknown").count()
```

```
Out[0]: 10
```

5b. passenger_id not actually unique

Bookings link to passengers by passenger_id, so checking that this field is actually unique before relying on it.

```
In [0]: dup_passenger_ids = passengers_clean.groupBy("passenger_id").count().filter(
dup_passenger_ids.count()
```

```
Out[0]: 36
```

Found 36 passenger_ids that each have 2-3 different rows - not exact duplicates, but conflicting versions of what looks like the same person (same name, age, and gender, but different email, phone, and aadhaar number). One case even has a corrupted name entry (first and last name merged together with a blank last name field).

The name, age, and gender are consistent across each person's duplicate rows, so those facts aren't actually in question - only contact details differ, and none of the dashboard KPIs use email, phone, or aadhaar directly. So keeping one clean row per passenger_id, preferring whichever row has a complete last name, so the corrupted entry doesn't get kept by accident. This has to happen before masking last_name below, since after masking every value looks like "X****" and the real null information would be lost.

```
In [0]: w = Window.partitionBy("passenger_id").orderBy(F.col("last_name").isNull()).a

passengers_deduped = (
    passengers_clean
    .withColumn("rn", F.row_number().over(w))
    .filter("rn = 1")
    .drop("rn")
```



```
)

passengers_deduped.count(), passengers_deduped.select("passenger_id").distir
```

Out [0]: (1000, 1000)

If both numbers above match, passenger_id is now a real unique key - no more conflicting rows.

Step 6: Protecting sensitive passenger data (PII masking)

Not all sensitive fields need the same treatment - using three different techniques depending on how the field is used and how sensitive it is:

- Aadhaar ID and passport number are government ID numbers - never needed in plain text for any analysis, so these get one-way hashed. Even the person running this notebook can't reverse it back to the original value.
- Email and phone are useful in a limited way (like checking two records are the same customer), so instead of hashing, partially masking them - keeping a small visible part, hiding the rest.
- Date of birth and emergency contact details aren't needed for any KPI at all, so rather than mask them, dropping them from the analysis-ready data entirely. No reason to carry data forward that nothing actually uses.

This is on top of the storage containers already being set to private access, so only people with proper Azure permissions can even reach this data in the first place.

```
In [0]: passengers_secure = (
    passengers_deduped
    .withColumn("aadhaar_id", F.sha2(F.col("aadhaar_id").cast("string"), 256))
    .withColumn("email", F.concat(F.substring("email", 1, 2), F.lit("***@")),
    .withColumn("phone", F.concat(F.lit("XXXXXX"), F.substring("phone", -4,
    .withColumn("last_name", F.concat(F.substring("last_name", 1, 1), F.lit(
    .drop("date_of_birth")
)

passengers_secure.show(5, truncate=False)
```

passenger_id	first_name	last_name	age	gender	email	phone	adhaar_id
P1000	Vivaan	C***	52	F	vi***@gmail.com	XXXXXX3790	99466baa9b8fe69a6f31db34c87d50f0e9a488eda788f3fdce1c00e09a316285
P1001	Krishna	R***	15	M	kr***@hotmail.com	XXXXXX2297	6d05ccbeb5011dd59948b882fff86e1a6f7fc0cf41f6598b385f58558830ac18
P1002	Myra	N***	72	M	my***@outlook.com	XXXXXX5092	3e24ac79e2c956797c22051676644d26204cc4de0f203e1bf3aec0d9ea8e0d3f
P1003	Myra	M***	61	F	my***@hotmail.com	XXXXXX7151	4e880723014ee24b39f0213150496f3efb9faa3f62d335c2248452b7c6ddc7e6
P1004	Saanvi	B***	21	M	sa***@outlook.com	XXXXXX5113	3ca09c31f18da71ad2bc39cb3cbb387c38a9cd5ad37239eb8ca5ac19c8c18ac1

only showing top 5 rows

```
In [0]: bookings_secure = (
    bookings_clean
    .withColumn("passport_number", F.sha2(F.col("passport_number").cast("str
    .drop("emergency_contact_name", "emergency_contact_phone")
    )

    bookings_secure.show(5, truncate=False)
```

booking_id	passenger_id	flight_id	booking_date	status	passport_number
B1000	P1591	AI192	2025-06-14 11:37:36.951	CANCELLED	59e7e7738c8711b2f49be2c355b25b9bd41b780392aa2151796e8b52aaa0a056
B1001	P1803	6F026	2025-11-02 11:37:36.951	CANCELLED	6a95f336b866dc4d580b61301c1a96016a5ec24e056b4062fcb7eb48c0e14578
B1002	P1083	SJ010	2025-08-25 11:37:36.951	CANCELLED	8d413701d70b517478f82789245fa033726307ec10b14c2c18fc6f6deb940931
B1003	P1364	AI069	2025-12-30 11:37:36.951	CONFIRMED	19c4a665e8bc1266b54aee9a9925f0d4179044fbc7be190242d1b89272a91bff
B1004	P1885	UK003	2025-10-02 11:37:36.951	PENDING	d9157c803314e083e13c0307a6e506c2beaa8c6e96140c318f3a5626acdbf658

only showing top 5 rows

Step 7: Joining bookings to flights

Before joining, checking for a problem the 6F250 issue could cause: if a booking references flight_id 6F250, a normal join would match it to both of the two different flights that share that ID, since there's nothing else to tell them apart. That would silently double that booking and throw off the numbers.

```
In [0]: fact_check = bookings_secure.join(flights_clean, on="flight_id", how="left")

ambiguous = fact_check.groupBy("booking_id").count().filter("count > 1")
ambiguous.show()
```

```
+-----+-----+
|booking_id|count|
+-----+-----+
|      B1294|    2|
|      B1235|    2|
+-----+-----+
```

Confirmed - 2 bookings can't be reliably matched to one specific flight, since the source data gives no way to tell which of the two 6F250 flights they actually belong to. Rather than guess (which could silently attach a booking to the wrong flight, wrong route, wrong duration), keeping these 2 bookings aside separately as "unresolved" - still valid bookings for booking-level numbers like revenue, just excluded from anything that needs a specific flight (like route or duration analysis).

```
In [0]: ambiguous_ids = [row.booking_id for row in ambiguous.collect()]
fact_bookings = fact_check.filter(~fact_check.booking_id.isin(ambiguous_ids))
fact_unresolved = fact_check.filter(fact_check.booking_id.isin(ambiguous_ids))

print("clean joined rows:", fact_bookings.count())
print("unresolved bookings set aside:", fact_unresolved.count())
```

```
clean joined rows: 998
unresolved bookings set aside: 4
```

One more check: when the invalid SJ192 flight was removed earlier, any booking that referenced that flight_id would now have no match in the flights table at all. This wouldn't get caught by the check above, since it only matches 0 flights, not more than 1 - it needs its own check.

```
In [0]: dangling = fact_bookings.filter(fact_bookings.flight_row_id.isNull())
dangling.count()
```

```
Out[0]: 1
```

Found 1 booking (B1636) referencing the flight that was removed earlier for having an invalid negative duration. Since that flight's data can't be trusted, there's no valid route

or duration to attach to this booking either. Same approach as the ambiguous bookings above - keeping this one aside rather than leaving it silently sitting in the main table with blank flight data.

```
In [0]: fact_unresolved = fact_unresolved.unionByName(dangling, allowMissingColumns=
fact_bookings = fact_bookings.filter(fact_bookings.flight_row_id.isNotNull())

print("fact_bookings count:", fact_bookings.count())
print("total unresolved bookings:", fact_unresolved.count())
```

```
fact_bookings count: 997
total unresolved bookings: 5
```

That's the full set of unresolved bookings now - the 2 from the 6F250 ambiguity (4 rows, since each matched twice) plus this 1 dangling one. Keeping them together in one place instead of losing track of them.

Step 8: Joining in payment totals

Using the aggregated payments table from Step 4c (one row per booking), so this join can't cause the same kind of duplication problem.

```
In [0]: fact_bookings = fact_bookings.join(payments_agg, on="booking_id", how="left"
fact_bookings.count())
```

```
Out[0]: 997
```

Step 9: Building the star schema

Organizing everything into a proper star schema instead of one wide table - a central fact_bookings table holding the keys and booking-level numbers, with separate dimension tables for flight details and passenger details. This avoids repeating the same flight information across every booking row, and matches standard data modeling practice.

dim_flight also gets one more column here: a flag for flights whose duration is unusually far off from the average duration on that same route (more than 2 standard deviations away). There's no "scheduled time" field in this data to calculate a traditional delay, so this is used as a stand-in anomaly signal instead - a flight taking much longer or shorter than others on the same route is worth a second look operationally.

```
In [0]: dim_flight = flights_clean.select(
    "flight_row_id", "flight_id", "airline", "source", "destination",
    "departure_time", "arrival_time", "duration_minutes", "is_overnight"
)

route_stats = dim_flight.groupBy("source", "destination").agg(
    F.avg("duration_minutes").alias("route_avg"),
```

```

    F.stddev("duration_minutes").alias("route_stddev")
  )

dim_flight = dim_flight.join(route_stats, on=["source", "destination"]).with
  "is_duration_anomaly",
  F.abs(F.col("duration_minutes") - F.col("route_avg")) > 2 * F.col("route
).drop("route_avg", "route_stddev")

dim_flight.filter("is_duration_anomaly = true").count()

```

Out[0]: 1

```

In [0]: dim_passenger = passengers_secure.select(
    "passenger_id", "first_name", "last_name", "age", "gender",
    "email", "phone", "aadhaar_id"
  )

fact_bookings = fact_bookings.select(
    "booking_id", "passenger_id", "flight_row_id", "booking_date", "status",
    "seat_number", "passport_number", "total_amount", "payment_count", "mult
  )

dim_flight.count(), dim_passenger.count(), fact_bookings.count()

```

Out[0]: (1004, 1000, 997)

This is now a real star schema - fact_bookings joins to dim_flight through flight_row_id, and to dim_passenger through passenger_id, and both keys are guaranteed unique on the dimension side, so no relationship built on top of this in Power BI can silently fan out.

Step 10: One flattened table for Power BI

The star schema above is the correct way to model this data, and it's what's documented in the architecture/data model writeup. For actually building the dashboard, joining everything into one flat table as well - simpler to work with in Power BI than managing relationships between three tables, especially without much prior Power BI experience. Also adding one more useful column here: how many days in advance the booking was made before the flight.

```

In [0]: gold = (
    fact_bookings
    .join(dim_flight, on="flight_row_id", how="left")
    .join(dim_passenger.select("passenger_id", "first_name", "last_name", "a
  )

gold = gold.withColumn(
    "booking_lead_time_days",
    (F.unix_timestamp("departure_time") - F.unix_timestamp("booking_date"))
  )

gold.count()

```

Out[0]: 997

```
In [0]: gold.filter(gold.booking_lead_time_days < 0).count()
```

Out[0]: 0

A negative value here would mean a booking got recorded as happening after its own flight already departed, which wouldn't make sense - a good sanity check to run before trusting this column on the dashboard.

Step 11: Exporting the Gold layer

Writing out the star schema tables plus the flat table, ready to be downloaded and loaded into Power BI.

```
In [0]: gold_path = "/Volumes/asg_airlines_databricks/default/raw_data/gold_output"
dbutils.fs.mkdirs(gold_path)

fact_bookings.toPandas().to_csv(f"{gold_path}/fact_bookings.csv", index=False)
dim_flight.toPandas().to_csv(f"{gold_path}/dim_flight.csv", index=False)
dim_passenger.toPandas().to_csv(f"{gold_path}/dim_passenger.csv", index=False)
gold.toPandas().to_csv(f"{gold_path}/gold_flat_table.csv", index=False)

print("exported 4 files to", gold_path)
```

exported 4 files to /Volumes/asg_airlines_databricks/default/raw_data/gold_output